

my ACM template

Contents

my ACM template	1
1. general	2
2. Graph theory	3
2.1. basic build graph	3
2.2. tarjan SCC	3
2.3. tarjan eDCC	3
2.4. tarjan vDCC	4
2.5. LCA	4
3. Math	5
3.1. Number theory	5
3.1.1. Euler sieve	5
3.1.2. Eratosthenes sieve	5
3.1.3. gcd	5
3.1.4. exgcd	5
3.1.5. lowbit	5
3.1.6. popcount	5
3.1.7. $\varphi(n)$	5

1. general

缺省源

```
#include <bits/stdc++.h>
#define pb push_back
#define vc vector
#define fi first
#define se second
#define all(x, n) (x) + 1, (x) + 1 + n
#define NL "\n"
#define NN " "
using namespace std;
typedef long long ll;
typedef vc<ll> vi;
typedef pair<ll, ll> PII;
typedef vc<PII> vpII;
template<class T, class S>
bool chmax(T& x, S y){ return x<y?x=y,1:0; }
template<class T, class S>
bool chmin(T& x, S y){ return x>y?x=y,1:0; }
// #define CF
// #define int long long
// =====

void solve() {
}
// =====

signed main() {
    ios::sync_with_stdio(0);
    cin.tie(0), cout.tie(0);
    cout.setf(ios::fixed), cout.precision(5);
    int T = 1;
#ifdef CF
    cin >> T;
#endif // CF
    while (T--) solve();
    return 0;
}
```

2. Graph theory

2.1. basic build graph

```
vpil e[N];
void add(int a,int b,int c){e[a].pb({b,c});}
// =====
int idx=0,h[N],ne[M],to[M],w[M];
void add(int a,int b,int c){
    w[++idx]=c,to[idx]=b,ne[idx]=h[a],h[a]=idx;
}
```

2.2. tarjan SCC

有向图中的 scc 要求当前点能够到达所有点，并且其他点能到当前点，于是利用这个思想，在 dfs 遍历的时候按照时间戳压栈，那么上面的点就是他能到的点（如果维护一定性质的话）直到遍历完，如果发现当前点就是这个连通块的最高的点（也就是 dfn 最小的点），那就把这个 scc 中的点全部出栈。scc 缩点之后，得到的是拓扑图。

```
vi e[N];
int dn = 0, dfn[N], low[N], stc[N], top = 0;
int col[N], cn = 0, sz[N];

void scc(int id) {
    low[id] = dfn[id] = ++dn;
    stc[++top] = id, ins[id] = 1;
    for (int it : e[id]) {
        if (!dfn[it]) {
            scc(it), chmin(low[id], low[it]);
        } else if (ins[it]) chmin(low[id], dfn[it]);
    }
    if (low[id] == dfn[id]) {
        cn++;
        int x;
        do {
            col[x = stc[top--]] = cn, ins[x] = 0, sz[cn]++;
        } while (x != id);
    }
}
```

2.3. tarjan eDCC

解决方法是记录边的编号。对于 vector，在 push_back 时将当前边的编号一并压入。对于链式前向星，使用成对变换技巧：初始化 cnt = 1，每条边及其反边在链式前向星中存储的编号分别为 2k 和 2k+1，将当前边编号异或 1 即得反边编号。时间复杂度 $O(n+m)$ 边双缩点之后，得到的是连通分量作为节点的树。

```
int dfn[N], low[N], dn = 0;
int stc[N], top = 0;
int cn = 0, col[M];

void form(int id) {
    cn++;
    for (int x = 0; x != id; x = stc[top--]) col[x] = cn;
}

void dcc(int id, int eid) {
    stc[++top] = id, dfn[id] = low[id] = ++dn;
    for (auto _ : e[id]) {
        if (_.se == eid) continue;
    }
}
```

```

    int it = _.fi;
    if (!dfn[it]) {
        dcc(it, _.se);
        chmin(low[id], low[it]);
        if (low[it] > low[id]) form(it);
    } else chmin(low[id], dfn[it]);
}
if (!eid) form(id);
}

```

2.4. tarjan vDCC

还不会呜呜呜

2.5. LCA

倍增法，先预处理深度和倍增祖先，然后先跳到一起，再一起跳。时间复杂度 $O(n)$ 预处理， $O(\log n)$ 查询

```

int dep[N], anc[N][20];
namespace ZX { // zu xian
void bfs(int root) {
    queue<int> q;
    dep[root] = 1, q.push(root);
    while (q.size()) {
        int id = q.front();
        q.pop();
        for (auto it : e[id]) {
            if (dep[it]) continue;
            dep[it] = dep[id] + 1;
            anc[it][0] = id;
            rep(i, 1, 19) anc[it][i] = anc[anc[it][i - 1]][i - 1];
            q.push(it);
        }
    }
}

int lca(int x, int y) {
    if (dep[x] < dep[y]) swap(x, y);
    per(i, 19, 0) if (dep[anc[x][i]] >= dep[y]) x = anc[x][i];
    if (x == y) return x;
    per(i, 19, 0) if (anc[x][i] != anc[y][i]) x = anc[x][i], y = anc[y][i];
    return anc[x][0];
}
} // namespace ZX

```

3. Math

3.1. Number theory

3.1.1. Euler sieve

线性筛，时间复杂度 $O(n)$

```
vi primes;
bool st[N];
void ss(int maxn){
    for(int i=2;i<=maxn;i++){
        if(!st[i]) primes.pb(i);
        for(int j:primes){
            if(j>maxn/i) break;
            st[i*j]=1;
            if(i%j==0) break;
        }
    }
}
```

3.1.2. Eratosthenes sieve

埃式筛，时间复杂度 $O(n \log \log n)$ ，修改内层循环可以做区间筛

```
vi primes;
bitset<N> st;
void ass(int maxn){
    for(int i=2;i<=maxn;i++){
        if(st[i]) continue;
        primes.pb(i);
        for(int j=i*2;j<=maxn;j+=i) st[j]=1;
    }
}
```

3.1.3. gcd

$(a, b) = (a, b - a) = (b, a - b)$ ，时间复杂度 $O(\log \min\{a, b\})$

```
ll gcd(ll x, ll y){ return y ? x : gcd(y, x%y); }
```

也可以，cpp 内置 `__gcd(x, y)`

3.1.4. exgcd

方程 $ax + by = \gcd(a, b)$ 的一组特解，通解是 $(x, y) = (x_0 + k \cdot \frac{b}{(a, b)}, y_0 - k \cdot \frac{a}{(a, b)}), k \in \mathbb{Z}$

```
void exgcd(int a, int b, int &x, int &y) {
    if(!b) return x = 1, y = 0, void();
    exgcd(b, a % b, y, x), y -= a / b * x;
}
```

3.1.5. lowbit

```
ll lowbit(ll x){ return x&(-x); }
```

3.1.6. popcount

```
ll count(ll x){ return __builtin_popcountll(x); }
int count(int x){ return __builtin_popcount(x); }
```

3.1.7. $\varphi(n)$

欧拉函数 $\varphi(n) = \sum_{i=1}^n [\gcd(i, n) = 1]$.

由容斥原理可推出： $n = p_1^{\alpha_1} \cdot p_2^{\alpha_2} \cdots p_j^{\alpha_j} \Rightarrow \varphi(n) = n \cdot \prod_{i=1}^j (1 - p_i^{-1}) = (p_1 - 1)(p_2 - 1) \cdots (p_j - 1) \cdot p_1^{\alpha_1-1} \cdot p_2^{\alpha_2-1} \cdots p_j^{\alpha_j-1}$. 且若 p 为素数，有 $\varphi(p) = p - 1$.

可以在 $O(\log n)$ 求单点欧拉函数，可以在 $O(n)$ 递推求 1 到 n 欧拉函数.

```
// $O(\log n)$ 求单点欧拉函数
ll get_phi(ll x){
    ll phi=1;
    for(ll i=2;i<=x/i;i++){
        if(x%i==0){
            phi*=(i-1);
            x/=i;
            while(x%i==0) phi*=i,x/=i;
        }
    }
    if(x>1) phi*=(x-1);
    return phi;
}

// $O(n)$ 递推求 $1$ 到 $n$ 欧拉函数
bool st[N];
vi primes;
ll phi[N];
void get_phi(ll maxn){
    phi[1]=1;
    for(ll i=2;i<=maxn;i++){
        if(!st[i]) primes.pb(i),phi[i]=i-1;
        for(ll j:primes){
            if(j>maxn/i) break;
            st[i*j]=1;
            if(i%j==0) { phi[i*j]=phi[i]*j; break; }
            phi[i*j]=phi[i]*(j-1);
        }
    }
}
```